

Memory Allocation Functions in C



malloc()

Stands for Memory
Allocation

Allocates single block of
requested memory



calloc()

Stands for Memory
Allocation

Allocates multiple block of
requested memory



realloc()

Stands for Memory
Allocation

Changes memory size that
is already allocated



free()

Stands for Memory
Allocation

Releases memory that is no
longer needed

💡 Key Concepts

These four functions form the core of dynamic memory management in C programming, allowing developers to allocate, reallocate, and free memory during program execution.



1/6

📦 Malloc() Stands for Memory Allocation

The `malloc()` function allocates single block of requested memory at runtime. This function reserves a block of memory of given size and returns a pointer of type void. This means that we can assign it to any type of pointer using typecasting. It doesn't initialize memory at execution time, so it has garbage value initially. If it fails to locate enough space (memory) it returns a NULL pointer.



Syntax:

```
ptr=(cast-type*)malloc(byte-size)
```



Example:

```
int *x;  
x = (int*)malloc(100 * sizeof(int)); //memory space allocated to variable x  
free(x); //releases memory allocated to variable x
```

This statement will allocate either 200 or 400 according to size of int 2 or 4 bytes respectively and the pointer points to address of first byte of memory.



Complete Example:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int num, i, *ptr, sum = 0;
    printf("Enter number of elements: ");
    scanf("%d", &num);
    ptr = (int*) malloc(num * sizeof(int)); //memory allocated using malloc
    if(ptr == NULL)
    {
        printf("Error! memory not allocated.");
        exit(0);
    }
    printf("Enter elements of array: ");
    for(i = 0; i < num; ++i)
    {
        scanf("%d", ptr + i);
        sum += *(ptr + i);
    }
    printf("Sum = %d", sum);
    free(ptr);
    return 0;
}
```



Calloc() Stands for Memory Allocation

Calloc() is another memory allocation function that is used for allocating memory at runtime. calloc function is normally used for allocating memory to derived data types such as arrays and structures. The calloc() function allocates multiple block of requested memory.

It initially initialize (sets) all bytes to zero. If it fails to locate enough space (memory) it returns a NULL pointer. The only difference between malloc() and calloc() is that, malloc() allocates single block of memory whereas calloc() allocates multiple blocks of memory each of same size.



Syntax:

```
ptr = (cast-type*)calloc(n/number, element-size);
```

calloc() required 2 arguments of type count, size-type. Count will provide number of elements; size-type is data type size



Example:

```
int*arr;  
arr=(int*)calloc(10, sizeof(int)); // 20 byte  
char*str;  
str=(char*)calloc(50, sizeof(char)); // 50 byte
```



Structure Example:

```
struct employee  
{  
    char *name;  
    int salary;  
};  
typedef struct employee emp;
```

```
emp *e1;  
e1 = (emp*)calloc(30, sizeof(emp));
```



Complete Example:

```
#include <stdio.h>  
#include <stdlib.h>  
  
int main()  
{  
    int num, i, *ptr, sum = 0;  
    printf("Enter number of elements: ");  
    scanf("%d", &num);  
    ptr = (int*) calloc(num, sizeof(int));  
    if(ptr == NULL)  
    {  
        printf("Error! memory not allocated.");  
        exit(0);  
    }  
    printf("Enter elements of array: ");  
    for(i = 0; i < num; ++i)  
    {  
        scanf("%d", ptr + i);  
        sum += *(ptr + i);  
    }  
    printf("Sum = %d", sum);  
    free(ptr);  
    return 0;  
}
```

Realloc() Stands for Memory Allocation

Changes memory size that is already allocated to a variable. Or If previously allocated memory is insufficient or more than required, you can change previously allocated memory size using `realloc()`.

If memory is not sufficient for `malloc()` or `calloc()`, you can reallocate memory by `realloc()` function. In short, it changes memory size. By using `realloc()` we can create the memory dynamically at middle stage. Generally by using `realloc()` we can reallocation of memory. `Realloc()` required 2 arguments of type `void*`, `size_type`. `Void*` will indicates previous block base address, size-type is data type size. `Realloc()` will creates memory in bytes format and initial value is garbage.



Syntax:

```
ptr=realloc(ptr, new-size)
```



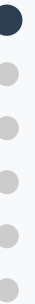
Example:

```
int *x;  
x=(int*)malloc(50 * sizeof(int));  
x=(int*)realloc(x,100); //allocated a new memory to variable x
```



Function Signature:

```
void*realloc(void*, size-type);
```





Example with Array:

```
int *arr;
arr=(int*)calloc(5, sizeof(int));
.....
.....
.....
arr=(int*)realloc(arr,sizeof(int)*10);
```



Complete Example:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr, i , n1, n2;
    printf("Enter size of array: ");
    scanf("%d", &n1);
    ptr = (int*) malloc(n1 * sizeof(int));
    printf("Address of previously allocated memory: ");
    for(i = 0; i < n1; ++i)
        printf("%u\t",ptr + i);
    printf("\nEnter new size of array: ");
    scanf("%d", &n2);
    ptr = realloc(ptr, n2);
    for(i = 0; i < n2; ++i)
```

```
printf("%u\t", ptr + i);  
return 0;  
}
```

4/6

Free() Stands for Memory Allocation

When your program comes out, operating system automatically release all the memory allocated by your program but as a good practice when you are not in need of memory anymore then you should release that memory by calling the function `free()`.

The memory occupied by `malloc()` or `calloc()` functions must be released by calling `free()` function. Otherwise, it will consume memory until program exit. Or Dynamically allocated memory created with either `calloc()` or `malloc()` doesn't get freed on its own. You must explicitly use `free()` to release space.





Syntax:

```
free(ptr);
```



Complete Example:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int num, i, *ptr, sum = 0;
    printf("Enter number of elements: ");
    scanf("%d", &num);
    ptr = (int*) malloc(num * sizeof(int)); //memory allocated using malloc
    if(ptr == NULL)
    {
        printf("Error! memory not allocated.");
        exit(0);
    }
    printf("Enter elements of array: ");
    for(i = 0; i < num; ++i)
    {
        scanf("%d", ptr + i);
        sum += *(ptr + i);
    }
    printf("Sum = %d", sum);
}
```

```
free(ptr);  
return 0;  
}
```



Memory Management Best Practice

Always free dynamically allocated memory when it's no longer needed to prevent memory leaks. The `free()` function is essential for proper memory management in C programs.



Conclusion

Memory allocation in C programming language is simple using static memory allocation which allocates memory during compile time or we can say before program execution and it also has another type known as dynamic memory allocation which allocates memory during run time or allocating memory during program execution which uses four different functions such as `malloc()`, `calloc()`, `free()` and `realloc()`.

There are different pros and cons of both methods.



malloc()

Allocates single block of memory with garbage values



calloc()

Allocates multiple blocks and initializes to zero



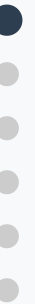
realloc()

Resizes previously allocated memory



free()

Releases allocated memory to prevent leaks



Key Takeaway

Dynamic memory allocation provides flexibility in managing memory resources during program execution, allowing programs to use memory more efficiently by allocating only what's needed when it's needed.